



ShadowFox

LEARN • CREATE • LEAD

CYBER SECURITY INTERNSHIP

FINAL TASK & PROJECT REPORT

Submitted By	MANOJ
Program ID	SF-PID-2026-W4884
Domain	Cyber Security
Internship Program	ShadowFox

Authorized internship and controlled lab activities documented for cybersecurity learning and evaluation.

TABLE OF CONTENTS

1. INTRODUCTION	3
2. INTERNSHIP OBJECTIVES	3
3. TOOLS AND TECHNOLOGIES	3
4. BEGINNER LEVEL – CYBER SECURITY TASKS	4
4.1 Task 1 – Port Scanning	4
4.2 Task 2 – Directory Enumeration	5
4.3 Task 3 – Network Traffic Analysis Using Wireshark	6
5. INTERMEDIATE LEVEL – CYBER SECURITY TASKS	7
5.1 Task 1 – VeraCrypt Encrypted File Analysis	7
5.2 Task 2 – PE Explorer Entry Point Analysis	9
5.3 Task 3 – Metasploit Reverse Shell Lab	10
5.4 Task 4 – Wireless Security Assessment	11
6. ADVANCED LEVEL – TRYHACKME BASIC PENTESTING	11
6.1 Objective	12
6.2 Lab Environment	12
6.3 Target Information	12
6.4 Reconnaissance	12
6.5 Port and Service Enumeration	12
6.6 Web / Application Enumeration	13
6.7 Other Service Enumeration	13
6.8 Vulnerability Identification	14
6.9 Initial Access	14
6.10 Privilege Escalation	15
6.11 Flags / Questions and Answers	16
6.12 Evidence and Screenshots	16
6.13 Attack Chain	17
6.14 Security Findings	17
6.15 Remediation / Patching Recommendations	17
6.16 Conclusion	17
7. CONSOLIDATED SECURITY FINDINGS	17
8. SECURITY RECOMMENDATIONS	18
9. SKILLS AND KNOWLEDGE GAINED	18
10. OVERALL CONCLUSION	19
11. REFERENCES	19
12. FINAL DECLARATION	19

Evidence integrity note. The three supplied reports are treated as the authoritative execution record. The Advanced report identifies five requested evidence figures as unavailable, and the Intermediate report contains no wireless-security execution evidence. No unsupported results have been fabricated.

1. INTRODUCTION

This final report consolidates the completed ShadowFox Cyber Security Internship work into one unified technical document. The work progresses from beginner-level reconnaissance and web traffic analysis, through intermediate encryption, file/executable analysis and controlled remote-access validation, to the Advanced Level TryHackMe Basic Pentesting room. The practical activities were carried out in deliberately vulnerable or authorized lab environments and are documented using the supplied evidence.

The Beginner work focused on identifying exposed services, enumerating web resources and inspecting HTTP traffic. The Intermediate work demonstrated protected-volume creation and mounting with VeraCrypt, hexadecimal file inspection, Windows PE analysis, network connectivity validation and a controlled Meterpreter session. The Advanced work demonstrated a structured penetration-testing workflow from reconnaissance and service enumeration through web discovery, SMB access, credential testing, Linux enumeration and SSH authentication.

The report distinguishes demonstrated lab activity from professional experience and records the methods, observations, limitations and evidence actually present in the submitted task reports rather than adding unsupported claims.

2. INTERNSHIP OBJECTIVES

- Perform network reconnaissance and identify exposed ports and services.
- Enumerate web directories and accessible application resources.
- Capture and analyze network traffic, including HTTP POST authentication traffic.
- Understand encryption, encrypted containers and password-based access control.
- Perform basic hexadecimal file analysis and Windows PE executable analysis.
- Validate network connectivity and Windows IP configuration in an authorized lab.
- Understand controlled payload delivery, listener configuration and remote-session validation in an isolated environment.
- Apply a structured penetration-testing methodology within an authorized TryHackMe lab.
- Assess exposed development resources, SMB access, credentials, SSH authentication and secret storage.
- Document technical findings, evidence limitations and practical remediation recommendations.

3. TOOLS AND TECHNOLOGIES

Kali Linux	Security-testing operating environment for reconnaissance, analysis and lab tooling.
Nmap	Open-port, service and version discovery.
Gobuster	Controlled web directory and resource enumeration.
Wireshark	Live packet capture, filtering and HTTP credential analysis.
Firefox / Web Browser	Interaction with authorized vulnerable web applications.
VeraCrypt	Creation, mounting and authenticated access to an encrypted container.
PE Explorer	Inspection of Windows PE headers and entry-point information.
Metasploit Framework	Controlled listener and session-validation workflow.
Meterpreter	Validation of the controlled remote-access session.
Local HTTP / Python HTTP server	Transfer of a lab executable within the authorized network.
SMB client / SMB enumeration	Assessment of anonymous SMB access and file retrieval.
Hydra	Authorized SSH credential testing in the TryHackMe lab.
SSH	Authenticated remote access to the lab Linux system.
John the Ripper	Password/passphrase-recovery concept referenced in the Advanced task; no separate crack-output evidence was supplied.
Standard Linux commands	Linux home-directory and file enumeration during the Advanced lab.

TryHackMe	Authorized training platform hosting the Basic Pentesting lab.

4. BEGINNER LEVEL – CYBER SECURITY TASKS

The Beginner Level consisted of three practical exercises against the authorized ShadowFox training target <http://testasp.vulnweb.com/>. The target IP observed in the report was **44.238.29.244**.

4.1 Task 1 – Port Scanning

Objective

Identify network-accessible TCP ports and determine the services and software versions associated with them.

Target and Tools

Target: **testasp.vulnweb.com** (44.238.29.244). Tool: **Nmap** from Kali Linux.

Methodology and Commands

A service/version scan was followed by an aggressive scan to gather additional HTTP and service information.

```
nmap -Pn -sV testasp.vulnweb.com
nmap -A testasp.vulnweb.com
```

Results

80/tcp	Open – HTTP; Microsoft IIS httpd 8.5; server header Microsoft-IIS/8.5; page title “Acuforum forums.”
HTTP method	TRACE was flagged by the -A scan as a potentially risky supported method.

Security Observation

The web server disclosed detailed version information through HTTP response headers. The report also notes the presence of TRACE, which in some configurations can contribute to cross-site tracing risk.

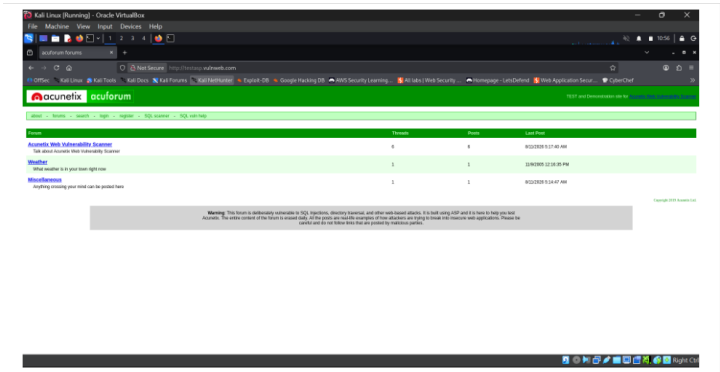


Figure 1: Target vulnerable web application used for the authorized security assessment.

Figure 1. Authorized target vulnerable web application used for the Beginner assessment.

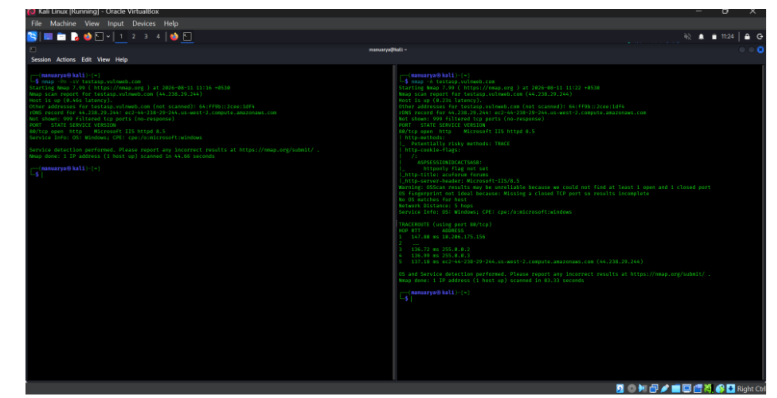


Figure 2: Nmap scan results identifying the open HTTP service and associated server information on the target.

Result

Figure 2. Nmap scan evidence identifying the open HTTP service and server information.

Conclusion

The exercise confirmed the exposed HTTP service on port 80 and the reported IIS 8.5 service information. A full 1–65535 port sweep was not conducted, so the result is limited to the scans performed.

4.2 Task 2 – Directory Enumeration

Objective

Identify directories and resources hosted by the target web application through controlled wordlist-based enumeration.

Tool and Command

Tool: **Gobuster** directory enumeration mode.

```
gobuster dir -u http://testasp.vulnweb.com/ -w /usr/share/wordlists/dirb/common.txt -t 5 --timeout 20s
```

Discovered Directories and Resources

Path	Status Code	Description
/_vti_cnf/	301	FrontPage configuration directory
/aspnet_client/	301	ASP.NET client-side resources
/avatars/	301	Forum avatar assets
/cgi-bin/	301	Redirect to /cgi-bin/
/cgi-bin/	403	Directory listing blocked
/html/	301	Static HTML content
/HTML/	301	Static HTML content (case variant)
/images/	301	Image assets
/Images/	301	Image assets (case variant)
/jscripsts/	301	JavaScript resources
/robots.txt	200	Robots exclusion file accessible
/t/	301	Short-named directory
/T/	301	Short-named directory (case variant)
/templates/	301	Application templates

The scan output also contained request timeouts. The source report therefore treats the listed paths as the resources for which a definitive response was received rather than claiming exhaustive coverage of the entire wordlist.

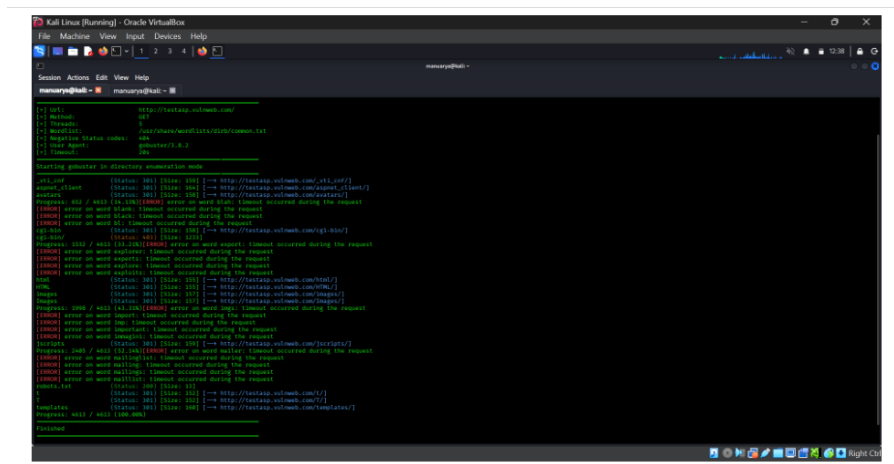


Figure 3: Gobuster directory enumeration results identifying confirmed directories and resources on the target web

Figure 3. Gobuster directory enumeration evidence showing confirmed web resources.

Conclusion

Gobuster mapped a set of accessible and partially accessible resources, including framework-related paths and a publicly readable robots.txt file.

4.3 Task 3 – Network Traffic Analysis Using Wireshark

Objective

Analyze HTTP traffic generated during an authorized login attempt and determine whether authentication credentials were transmitted without encryption.

Test Environment

Interface: **eth0** Local IPv4 observed: **10.206.175.48/24** Target IP: **44.238.29.244**.

Methodology and Traffic Capture

Wireshark was started on eth0 before interacting with the login page. A login attempt was submitted through Login.aspx, then the capture was filtered to isolate HTTP POST requests.

```
http.request.method == "POST"
```

Relevant Packet and Credential Analysis

The captured request was: **POST /Login.aspx?ReturnURL=%2FSearch%2Easp%3F HTTP/1.1** with **Content-Type: application/x-www-form-urlencoded**. The supplied form data showed the training credentials **tfUserName = admin** and **tfUPass = admin**.

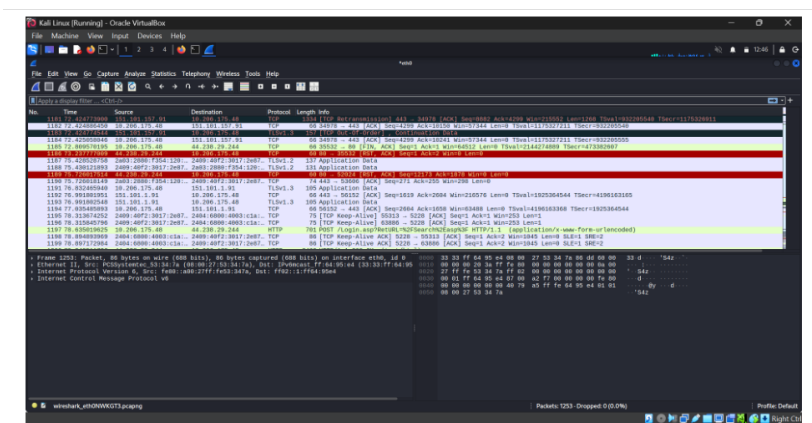


Figure 4: Wireshark packet capture showing HTTP traffic observed on the active eth0 interface during interaction with the target application.

Figure 4. Wireshark packet capture showing HTTP traffic on the active eth0 interface.

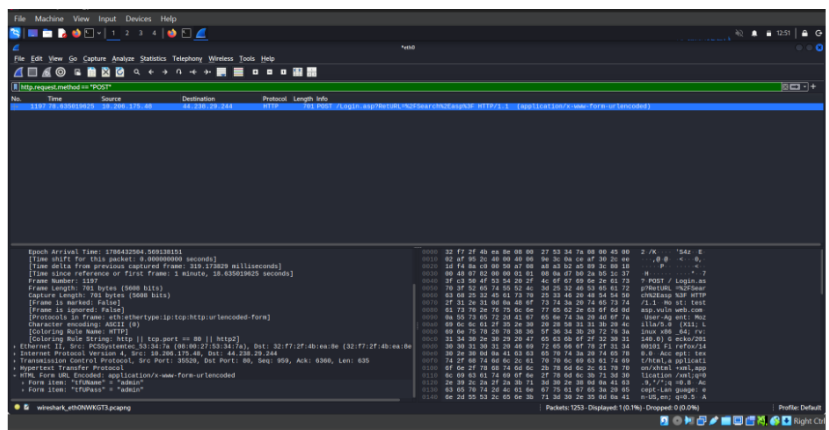


Figure 5: Wireshark analysis of the HTTP POST request showing credentials transmitted through the unencrypted connection.

Findings

Figure 5. Wireshark analysis of the HTTP POST request showing credentials in the unencrypted connection.

Security Implications

The credentials were transmitted over plain HTTP rather than HTTPS, making the submitted values visible in the captured request. The finding is limited to the authorized training application.

Recommended Mitigation

- Use HTTPS/TLS for all authentication traffic.
- Redirect HTTP requests to HTTPS automatically.
- Enable HSTS where appropriate.
- Avoid transmitting authentication credentials over plaintext HTTP.
- Apply secure cookie attributes and enforce proper session management.

Conclusion

The traffic analysis confirmed plaintext credential transmission over HTTP and demonstrated the security importance of enforcing HTTPS for authentication and session handling.

5. INTERMEDIATE LEVEL – CYBER SECURITY TASKS

The supplied Intermediate report documents practical activities covering file analysis, VeraCrypt encrypted-volume handling, PE Explorer analysis, network connectivity validation, Windows IP configuration, controlled payload generation, local HTTP transfer, Metasploit handler configuration and Meterpreter session validation.

5.1 Task 1 – VeraCrypt Encrypted File Analysis

Objective

Demonstrate basic file/hexadecimal inspection, encrypted-volume creation, password-based authentication and access to protected files.

Tools / Files Used

Kali Linux, a supplied file inspected at hexadecimal level, VeraCrypt on the Windows lab system, and a protected text file within the mounted volume.

Methodology

- Inspect the supplied file at a basic byte/hexadecimal level.
- Install VeraCrypt on the Windows lab system.
- Use the VeraCrypt Volume Creation Wizard to create a protected container.
- Authenticate to the selected protected volume with its password.
- Mount the volume and access the protected file through the assigned drive letter.

Password / Hash Analysis

The supplied Intermediate report does not document a separate password-hash cracking procedure for the VeraCrypt volume. It documents password-based authentication as the control required to mount the protected volume.

VeraCrypt Unlock Process and Secret Code

The supplied evidence shows the password prompt, successful mounting and access to a text file. The text file visibly contains the secret code: **never giveup**.

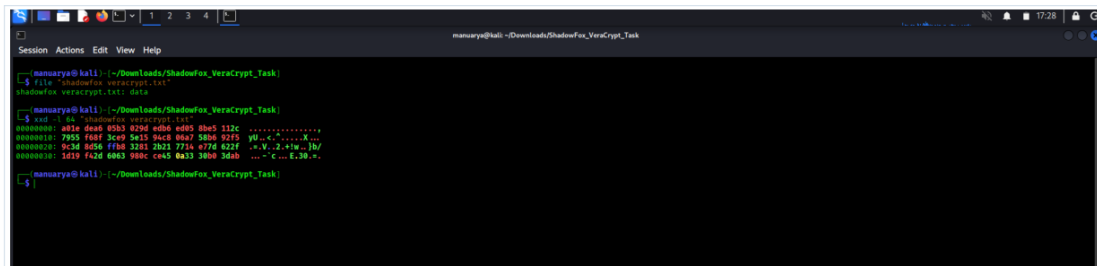


Figure 1: File Analysis Using Hex Dump

The screenshot shows a hexadecimal view of the supplied file, demonstrating basic low-level file inspection and the visibility of raw byte data.

Figure 6. File analysis using a hexadecimal dump.

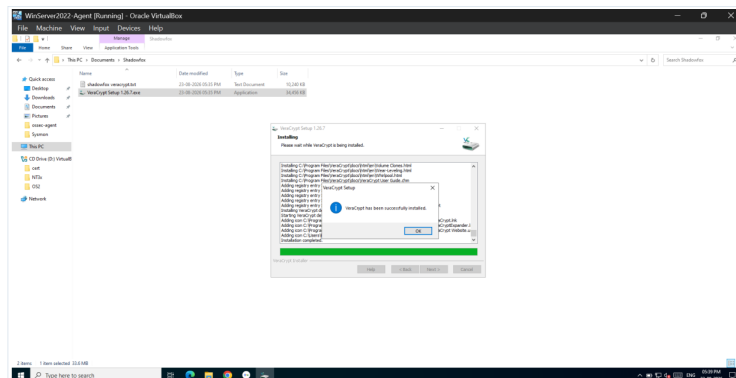


Figure 7. Successful VeraCrypt installation.

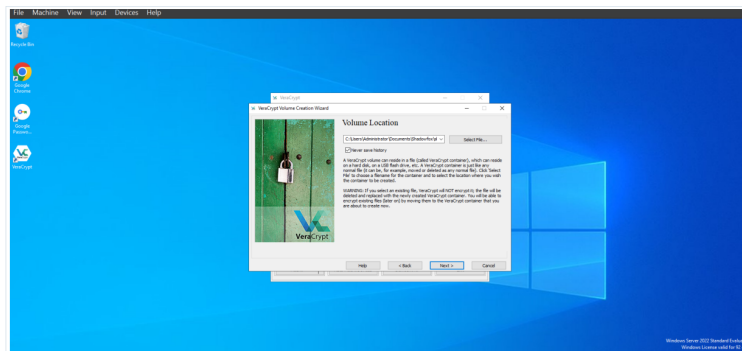


Figure 3: VeraCrypt Encrypted Volume Creation

Figure 8. VeraCrypt encrypted-volume creation workflow.

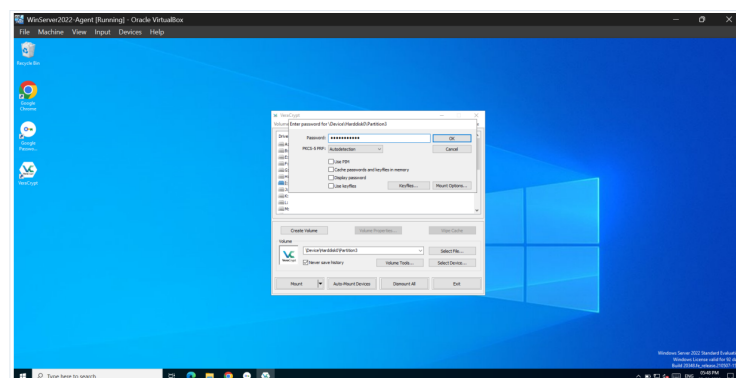


Figure 9. Password authentication for the protected volume.

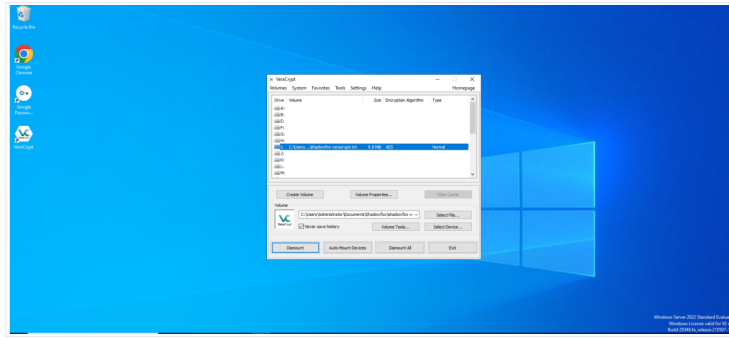


Figure 5: Encrypted Volume Successfully Mounted

The VeraCrypt main window shows the protected volume mounted and associated with a drive letter. The interface

Figure 10. Encrypted volume successfully mounted.

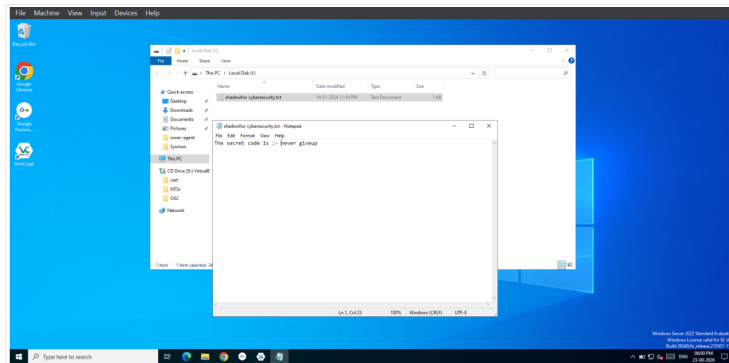


Figure 2. Accession Files from the Mounted Encountered Volume

Figure 11. Accessing the file from the mounted encrypted volume.

Security Observations

Encrypted storage protects confidentiality while the volume is not authenticated and mounted; strong authentication is required to complement the protection provided by encryption.

Conclusion

The task demonstrated the protected-storage workflow from low-level file inspection and VeraCrypt installation through authenticated mounting and access to the protected file.

5.2 Task 2 – PE Explorer Entry Point Analysis

Objective

Inspect the structure of a Windows executable and identify the entry-point value displayed by PE Explorer.

Executable and Tool

Executable analyzed: the VeraCrypt installer executable shown in the supplied evidence. Tool: **PE Explorer**.

Analysis Procedure

PE Explorer was installed and used to inspect PE header information, including entry-point and image/header fields.

Entry Point Address: 004237B0

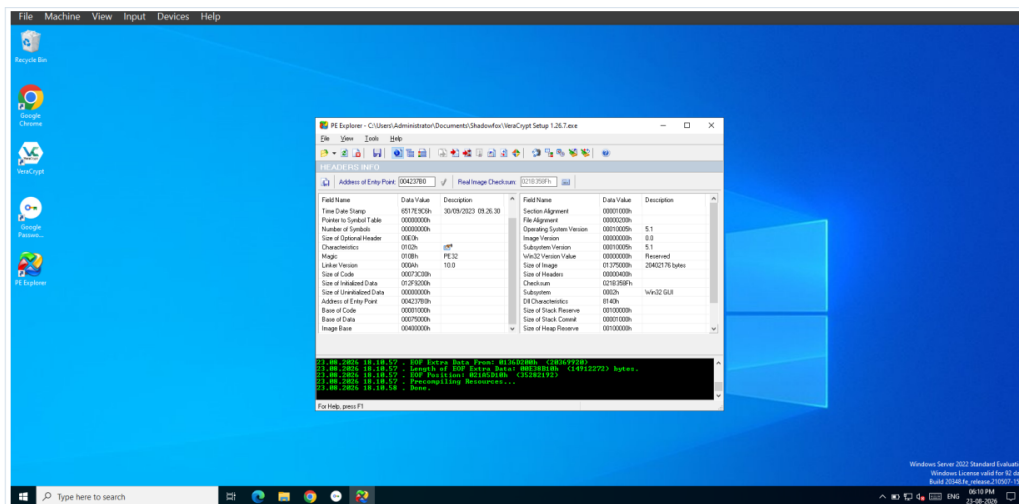


Figure 8: Windows Executable Analysis Using PE Explorer

Figure 12. PE Explorer analysis of the VeraCrypt installer; the displayed entry-point value is 004237B0.

Conclusion

The PE Explorer evidence demonstrates basic Windows PE structure analysis and records the entry-point value displayed in the supplied screenshot.

5.3 Task 3 – Metasploit Reverse Shell Lab

This activity is presented strictly as an authorized lab demonstration. The supplied report describes the workflow as isolated remote-access testing between authorized lab systems.

Objective and Lab Architecture

Validate connectivity, generate a controlled Windows x64 reverse-TCP test payload, transfer it over the lab network, configure a Metasploit handler and validate the resulting Meterpreter session.

Lab architecture: Kali Linux security-testing workstation → local HTTP transfer → authorized Windows lab endpoint → controlled reverse-TCP listener/session.

Methodology

- Verify reachability between the authorized lab systems using ICMP.
- Review the Windows IPv4 configuration, subnet mask and default gateway.
- Generate the lab payload as part of the controlled demonstration.
- Host the lab directory with a local HTTP server and observe the Windows host requesting the file.
- Configure Metasploit multi/handler with matching Windows x64 reverse-TCP test settings and a listening port.
- Start the handler and validate the Meterpreter session through system and user-context checks.

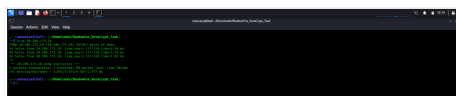


Figure 9: Network Connectivity Testing Between Lab Systems

The Kali terminal shows successful ICMP replies from the Windows lab host, with the displayed ping statistics reporting 0% packet loss.

Figure 13. Network connectivity testing between the authorized lab systems.

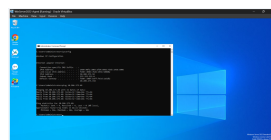


Figure 14. Windows IP configuration shown in the lab evidence.

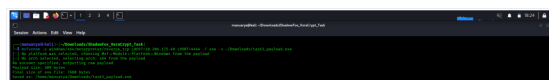


Figure 11: Controlled Payload Generation in the Lab

The Kali terminal shows that a Windows x64 reverse TCP test payload was generated successfully and saved as an executable file in the Downloads directory.

Figure 15. Controlled Windows x64 reverse-TCP payload generation in the lab.



Figure 16. Metasploit handler configuration for the controlled test.



Figure 13: Payload Delivery Through a Local HTTP Server
A local Python HTTP server is shown serving the Downloads directory. The log records an HTTP request from the Windows lab host, demonstrating local payload transfer over the lab network.

Figure 17. Payload delivery through a local HTTP server.

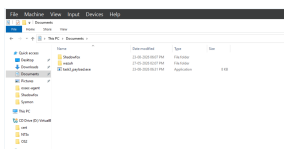


Figure 18. Generated executable present on the Windows lab system.

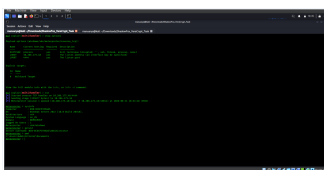


Figure 19. Successful controlled Meterpreter session with system and user-context validation.

Security Implications

The exercise illustrates how endpoint exposure, network communication, executable delivery and remote-session mechanisms interact. The source report emphasizes endpoint monitoring, prevention of suspicious executable activity, least privilege and explicit authorization.

Mitigation Recommendations

- Use EDR and antivirus controls to detect suspicious activity.
- Restrict unnecessary inbound and outbound network communication.
- Monitor unexpected executables, command shells and remote sessions.
- Apply least-privilege principles to users and service accounts.
- Keep Windows and security software patched and supported.
- Perform remote-access testing only with explicit authorization and defined scope.

Conclusion

The supplied evidence demonstrates successful controlled payload generation, local transfer, handler configuration and Meterpreter session validation within the authorized lab.

5.4 Task 4 – Wireless Security Assessment

Source-evidence limitation: The ShadowFox task structure supplied with this request specifies a wireless-security assessment, but the uploaded completed Intermediate report contains no wireless-assessment section, wireless screenshots, handshake evidence, wordlist evidence or password-recovery result. In accordance with the no-fabrication requirement, this report does not assert a deauthentication process, handshake capture, wordlist preparation or recovered password.

The task is retained in the unified structure as an evidence-status entry rather than being filled with unsupported technical details. A separate completed wireless report or the missing wireless evidence would be required to document that task accurately.

6. ADVANCED LEVEL – TRYHACKME BASIC PENTESTING

The selected Advanced Level task was the **TryHackMe Basic Pentesting** room. The supplied report documents an authorized training-machine assessment using Kali Linux and a sequence of reconnaissance, enumeration,

credential-testing and SSH-access steps.

6.1 Objective

Perform network reconnaissance, web discovery, information-disclosure analysis, SMB enumeration, authorized SSH credential testing, Linux user/home-directory enumeration, private-key analysis and final validation within the TryHackMe Basic Pentesting lab.

6.2 Lab Environment

Platform	TryHackMe
Attacker machine	Kali Linux
Target	TryHackMe Basic Pentesting lab machine
Tools	Nmap, Gobuster, Hydra, SMB client / SMB enumeration, SSH, John the Ripper and standard Linux commands
Reconnaissance address	10.48.141.61
Later SSH evidence address	10.146.147.11; the source report does not assume it is identical to the earlier address.

6.3 Target Information

The supplied reconnaissance evidence identifies the target as **10.48.141.61**. A later SSH screenshot visibly uses **10.146.147.11**; the source report deliberately keeps these observations separate.

6.4 Reconnaissance

Nmap was used to identify the reachable host and enumerate exposed network services. The evidence shows SSH, HTTP, SMB-related services, AJP and Tomcat.

6.5 Port and Service Enumeration

```
sudo nmap -sS -A -T5 10.48.141.61
```

22/tcp	SSH – OpenSSH 8.2p1 Ubuntu 4ubuntu0.13
80/tcp	HTTP – Apache httpd 2.4.41 (Ubuntu)
139/tcp	NetBIOS-SSN – Samba smbd 4
445/tcp	Microsoft-DS – Samba smbd 4
8009/tcp	AJP13 – Apache Jserv (Protocol v1.3)
8080/tcp	HTTP – Apache Tomcat/Coyote JSP engine 1.1; Apache Tomcat 9.0.7 shown

```

manuarya@kali: ~$ nmap -sV 10.48.141.61
Starting Nmap 7.99 (https://nmap.org) at 2020-08-28 11:29 +0530
Warning: 10.48.141.61 giving up on port because retransmission cap hit (2).
Nmap scan report for 10.48.141.61
Host is up (0.20s latency).
Not shown: 994 closed tcp ports (reset)
PORT      STATE SERVICE        VERSION
22/tcp    open  ssh            OpenSSH 8.7p1 Ubuntu aubuntu0.13 (Ubuntu Linux; protocol 2.0)
|_ ssh-hostkey:
|_ 1024 1024:22:1d:ad:c4:f3:be:ed:f8:10:30:73:b1:cb:00:10:00 (RSA)
|_ 256 256:cd:ab:87:df:76:17:df:9f:fa:fd:26:1e:11:f9:34 (ECDSA)
|_ 256 256:cd:ab:87:df:76:17:df:9f:fa:fd:26:1e:11:f9:34 (ECDSA)
80/tcp    open  http            Apache/2.4.41 ((Ubuntu))
|_ http-server-header: Apache/2.4.41 ((Ubuntu))
|_ http-title: Site does't have a title (text/html).
139/tcp   open  netbios-ssn     Samba smbd 4.
445/tcp   open  netbios-ssn     Samba smbd 4.
8080/tcp   open  http            Apache/2.4.41 ((Ubuntu))
|_ http-methods:
|_ Supported methods: GET HEAD POST OPTIONS
|_ 8080/tcp open http            Apache Tomcat/9.0.7
|_ http-titles: Apache Tomcat/9.0.7
|_ http-favicon: Apache Tomcat
Aggressive OS guesses: Linux 4.15 - 5.19 (90%), Linux 5.14 - 6.8 (92%), Linux 5.14 - 5.15 (93%), Linux 3.3 (92%), Linux 4.19 (92%), Linux 4.15 (92%), Linux 5.10 (91%), Ruckus ZoneFlex 8710 WAP (Linux 3.4) (91%), Linux 4.1 (91%), Linux 2.6.32 (91%)
No exact OS matches for host (test conditions non-ideal).
Network Distance: 3 hops
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Host script results:
|_ clock-view: 1d18h
|_ smb-time:
|_ date: 2020-08-28T06:01:30
|_ start-date: 0d0
|_ smb-security-mode:
|_ 3.1.1:
|_ Message signing enabled but not required
|_ smbstat: NetBIOS name: BASIC2, NetBIOS user: cunkmann, NetBIOS MAC: cunkmann (unknown)

TRACEROUTE (using port 21/tcp)
HOP RTT      0.00ms
1  341.89 ms 192.168.192.1
2  ...
3  621.89 ms 10.48.141.61

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .

```

Nmap identifies the lab host as reachable and shows multiple exposed services. The screenshot provides the

Figure 20. Nmap service enumeration evidence for the TryHackMe target.

6.6 Web / Application Enumeration

Gobuster was used to enumerate web content and identify a development directory.

```
gobuster dir -u http://10.48.141.61/ -w /usr/share/wordlists/gobuster/directory-list-2.3-medium.txt
```

The supplied output identified **/development/** with HTTP status 301, establishing the next web-investigation point.

Figure 2 - Web Directory Enumeration Using Gobuster

```

manuarya@kali: ~$ gobuster dir -u http://10.48.141.61/ -w /usr/share/wordlists/gobuster/directory-list-2.3-medium.txt
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Muehlbauer (@Firefart)

+ Url:      http://10.48.141.61/
+ Method:   GET
+ Threads:  10
+ Wordlist: /usr/share/wordlists/gobuster/directory-list-2.3-medium.txt
+ Negative Status codes: 404
+ User Agent: gobuster/3.8.2
+ Timeout:  10s

Starting gobuster in directory enumeration mode

development (Status: 301) [Size: 318] [→ http://10.48.141.61/development/]
Progress: 4028 / 228559 (1.83%)
manuarya@kali: ~$

```

Figure 21. Gobuster evidence identifying the development directory.

6.7 Other Service Enumeration

The development area was examined through exposed files. **dev.txt** contained internal implementation notes referencing Struts, SMB configuration and Apache setup. **j.txt** disclosed a credential-related clue stating that an **/etc/shadow** audit found a weak credential and that its hash had been cracked.

```

2018-04-23: I've been messing with that struts stuff, and it's pretty cool! I think it might be neat to host that on this server for, haven't made any real web apps yet, but I have tried that example you get to show off how it works (and it's the REST version of the example). Oh, and right now I'm using version 2.5.12, because other versions were giving me trouble. -K

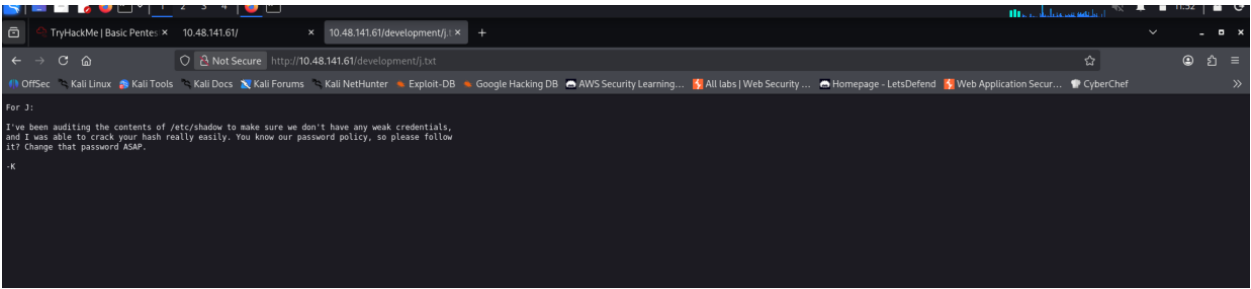
2018-04-22: SMB has been configured. -K

2018-04-21: I got Apache set up. Will put in our content later. -J

```

The development note exposes internal implementation details, including references to Struts, SMB configuration and Apache setup. This demonstrates why development artifacts should not be publicly accessible.

Figure 22. Information disclosure through dev.txt.



The exposed j.txt note states that an /etc/shadow audit found a weak credential and that its hash was cracked. This is a direct credential-security clue within the lab.

Figure 23. Credential-related clue identified in j.txt.

SMB enumeration was also demonstrated. The supplied evidence shows access to the Anonymous share and retrieval of **staff.txt**, whose note references Jan as a relevant username lead.

Figure 24. SMB Anonymous Access

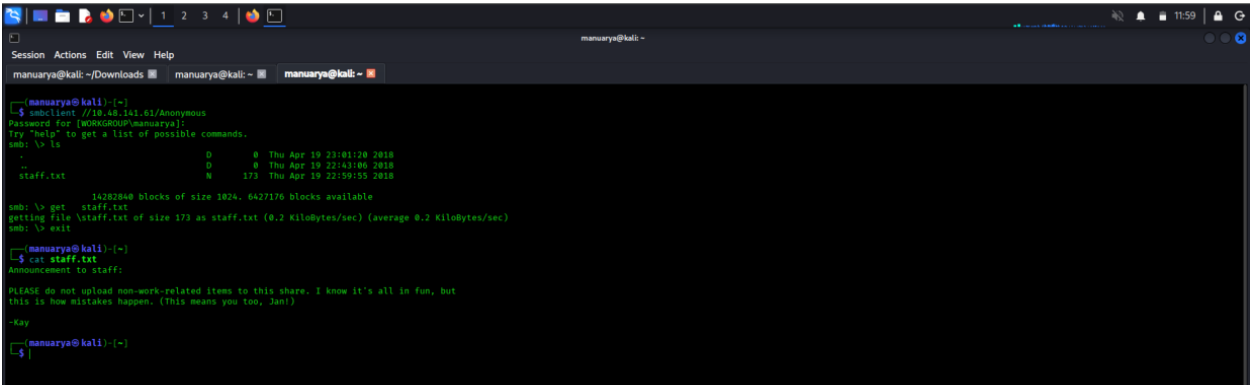


Figure 24. SMB anonymous access and retrieval of staff.txt.

6.8 Vulnerability Identification

Exposed development resources	Internal development notes can reveal implementation details and operational context.
Information disclosure	Public notes disclosed credential-related information and system-administration context.
SMB anonymous access	Unauthenticated access allowed file listing and retrieval, increasing information-exposure risk.
Weak / exposed credentials	Authorized Hydra testing demonstrated a valid SSH credential for jan.
Exposed SSH private key	An encrypted RSA private key was accessible in the kay user's .ssh directory during the lab workflow.
Weak secret storage	A password was stored in pass.bak, demonstrating unsafe local secret storage.

6.9 Initial Access

Hydra was used for authorized SSH credential testing in the TryHackMe lab and the evidence records a valid credential for the **jan** account.

```
hydra -l jan -P /usr/share/wordlists/rockyou.txt ssh://10.48.141.61
```

The supplied screenshot provides the username and password used for the subsequent lab access path.

Figure 8 - Authorized SSH Credential Testing Using Hydra

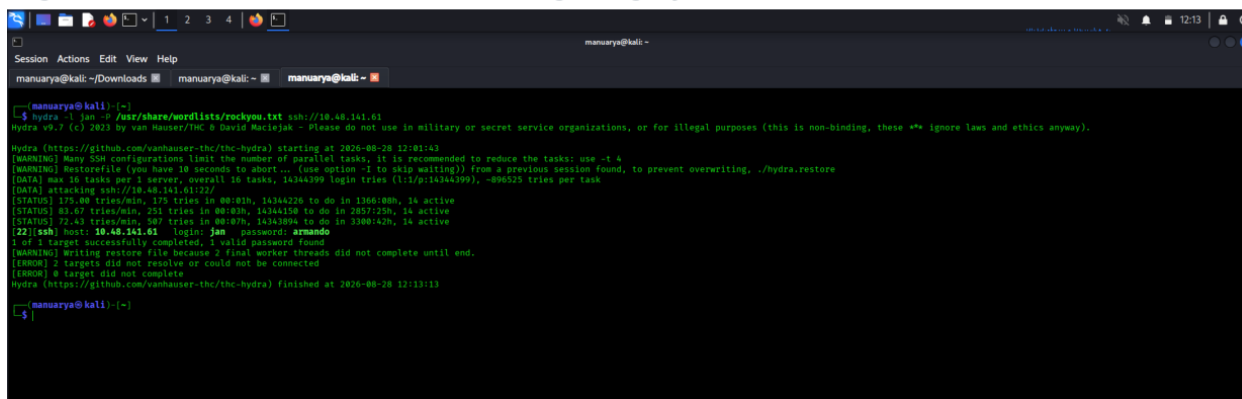


Figure 25. Authorized SSH credential testing using Hydra.

6.10 Privilege Escalation

No separate root/privilege-escalation result is asserted. The supplied report documents progression from the jan account to discovery of the kay user's home directory, an encrypted RSA private key and subsequent key-based SSH authentication as kay. The source report does not document a root shell or a distinct privilege-escalation result.

Figure 10 - Enumeration of the Kay User Home Directory

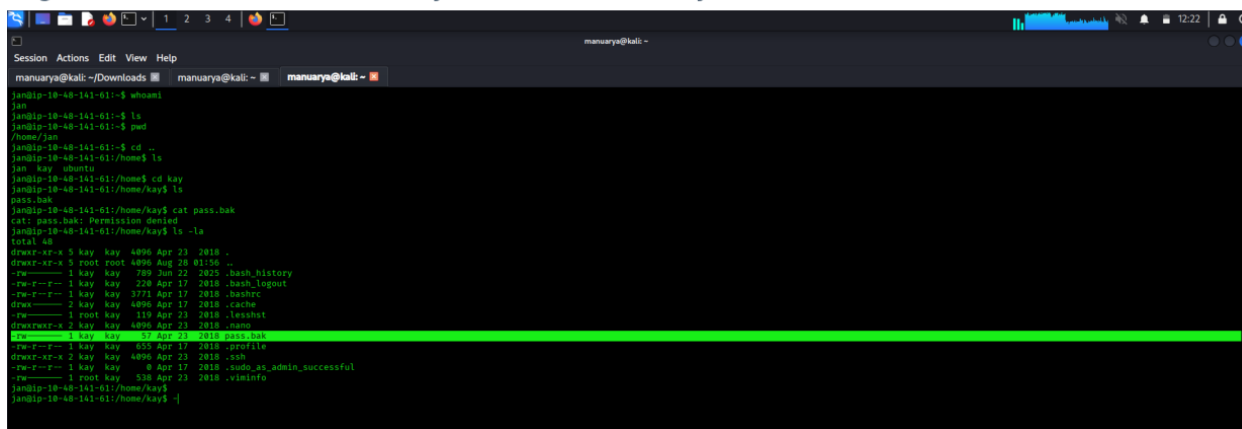


Figure 26. Enumeration of the kay user home directory.

/home/kay. The pass.wak file is present but initially inaccessible to the jain user, while the .ssh directory is visible in the listing.

Figure 12 - Discovery of the SSH Private Key

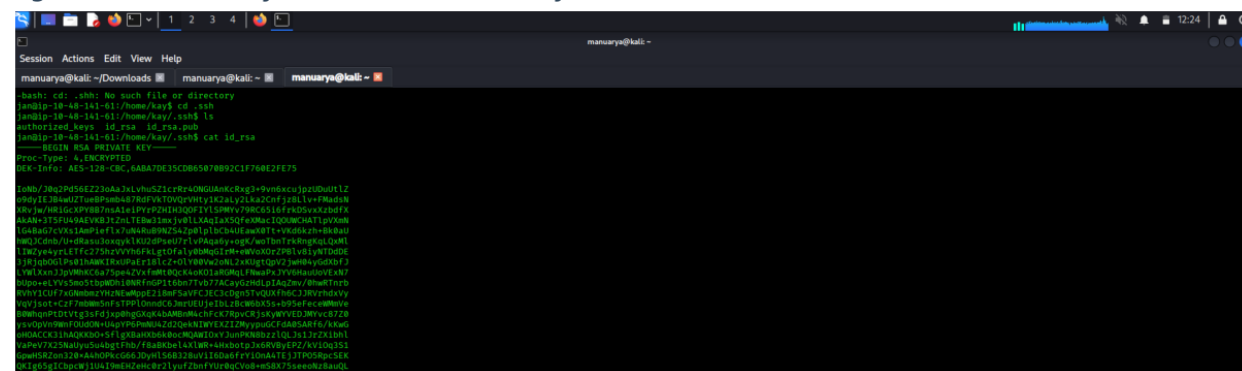


Figure 27. Discovery of the encrypted RSA private key in kay's .ssh directory.

Figure 14 - SSH Authentication Using the Recovered Key

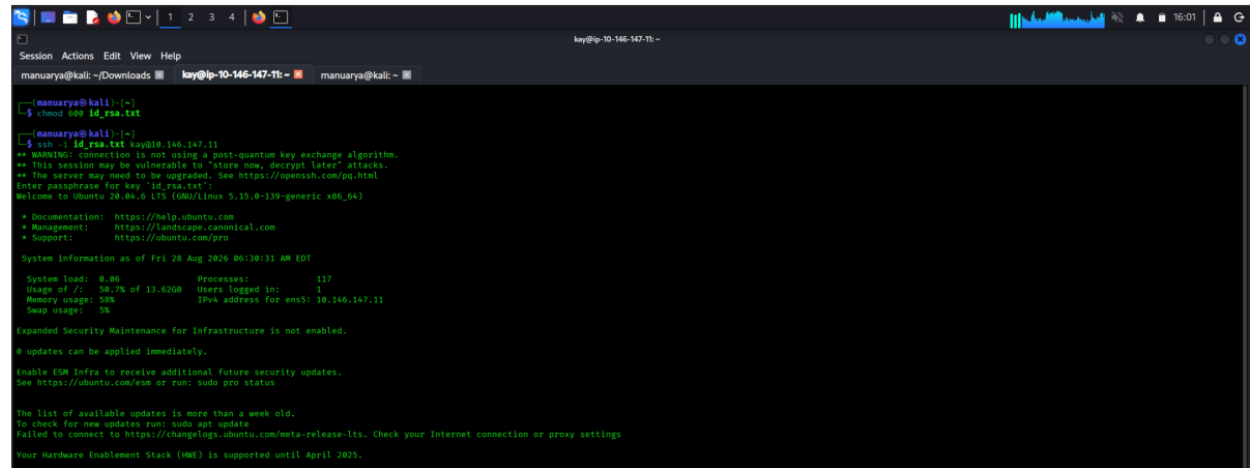


Figure 28. SSH authentication using the supplied key and access to the kay shell.

6.11 Flags / Questions and Answers

What is the name of the hidden directory on the web server?	development
What is the username?	jan
What is the password?	armando
What service do you use to access the server?	SSH
What is the name of the other user you found?	kay
If you have found another user, what can you do with this information?	The supplied evidence demonstrates that kay was used as a lead for further Linux/home-directory enumeration and subsequent key-based SSH access. The exact room answer text is not visible in the supplied screenshots.
What is the final password you obtain?	heresareallystrongpasswordthatfollowsthepasswordpolicy\$\$

The Advanced source report states that the individual answer fields were not visible in the completion screenshot. The table preserves only answers directly supported by the supplied report, including the qualified wording for the kay-user question.

6.12 Evidence and Screenshots

The supplied Advanced report explicitly identifies five requested figures that were not separately supplied: Figures 03, 06, 09, 11 and 13. Those missing figures are not recreated or represented as independent evidence.



The supplied screenshot demonstrates SSH authentication as kay using an id_rsa.txt key and then shows access to the kay shell. It also shows a lab password stored in pass.bak after login.

Figure 15 - TryHackMe Basic Pentesting Room Completion

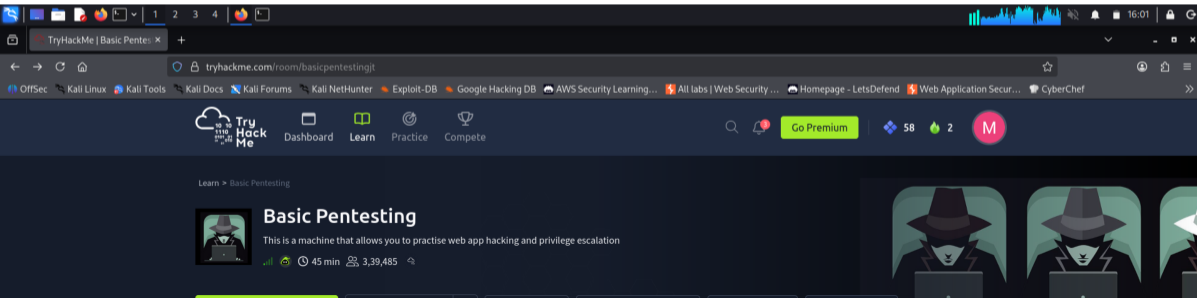
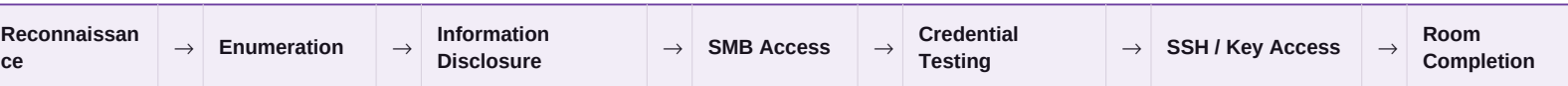


Figure 29. TryHackMe Basic Pentesting room completion evidence showing 100% completion.

6.13 Attack Chain



This flow reflects only the stages documented in the supplied evidence. No separate privilege-escalation stage is added because the source report does not document one.

6.14 Security Findings

Exposed development resources	Web application	Internal implementation and operational details may aid further enumeration.	Figure 22	Remove development/debugging/admin notes from public web paths.
Information disclosure	Web application	Public notes disclosed credential-related and administrative context.	Figure 23	Remove sensitive notes and test artifacts from exposed paths.
SMB anonymous access	SMB	Unauthenticated file listing/retrieval increases information exposure.	Figure 24	Disable anonymous access unless explicitly required; apply least privilege.
Weak / exposed credentials	SSH	A valid credential was discovered through authorized testing.	Figure 25	Enforce strong password policies and monitor authentication attempts.
Exposed SSH private key	Linux / SSH	Private-key exposure increases risk if key protection is weak.	Figure 27	Use strong key passphrases and restrictive filesystem permissions.
Weak secret storage	Linux filesystem	Password stored in pass.bak demonstrates unsafe secret storage.	Figure 28	Remove plaintext secrets and rotate exposed credentials.

6.15 Remediation / Patching Recommendations

- Remove sensitive development, debugging and administrative notes from publicly accessible web directories.
- Disable anonymous SMB access unless explicitly required and apply least-privilege share/file permissions.
- Enforce strong password policies and monitor authentication failures and repeated login attempts.
- Use strong, unique passphrases for SSH private keys and protect them with restrictive filesystem permissions.
- Rotate credentials exposed or recovered during security assessment.
- Restrict SSH exposure to trusted management networks where practical and use appropriate authentication controls.
- Monitor authentication logs and unusual access to sensitive files or user home directories.
- Apply least privilege to user accounts and prevent unnecessary cross-user home-directory access.
- Perform regular vulnerability assessments after configuration changes.
- Avoid deploying development/debugging files, notes or test artifacts to production-facing web paths.

6.16 Conclusion

The TryHackMe Basic Pentesting room provided practical exposure to an authorized penetration-testing workflow: reconnaissance, service enumeration, web discovery, information disclosure, SMB access, credential discovery, Linux enumeration and SSH authentication. The supplied evidence demonstrates how several small configuration and credential-management weaknesses can be chained within a training environment.

7. CONSOLIDATED SECURITY FINDINGS

Server version disclosure / risky HTTP method	Beginner web target	Detailed IIS information and TRACE support may increase attack-surface knowledge.	Beginner Figures 1–2	Reduce unnecessary banner disclosure and disable unnecessary HTTP methods.

Web resource exposure	Beginner web target	Framework/configuration paths and robots.txt reveal application structure.	Beginner Figure 3	Review public paths and remove unnecessary exposed resources.
Plaintext authentication traffic	Beginner web target	Credentials are visible in HTTP POST traffic.	Beginner Figures 4–5	Enforce HTTPS/TLS and secure session handling.
Weak authentication choice risk	Encrypted storage	Weak passwords can undermine otherwise strong encrypted storage.	Intermediate Figures 9–11	Use strong, unique passwords/passphrases.
Controlled remote-access exposure	Windows lab endpoint	Suspicious payload execution and remote sessions require strong endpoint controls.	Intermediate Figures 15–19	Use EDR/AV, network restrictions, monitoring and least privilege.
Exposed development resources	TryHackMe web service	Internal notes can aid further enumeration.	Advanced Figure 22	Remove development artifacts from public paths.
Information disclosure	TryHackMe web service	Credential-related notes can reveal attack-path information.	Advanced Figure 23	Remove sensitive notes and rotate exposed secrets.
SMB anonymous access	TryHackMe SMB	Unauthenticated access permits information retrieval.	Advanced Figure 24	Disable anonymous access and enforce least privilege.
Weak / exposed credentials	TryHackMe SSH	Valid SSH credentials can enable authenticated access.	Advanced Figure 25	Use strong passwords and monitor authentication attempts.
Exposed SSH private key	TryHackMe Linux/SSH	Key exposure creates additional authentication risk.	Advanced Figure 27	Protect private keys and require strong passphrases.
Weak secret storage	TryHackMe Linux filesystem	Password stored in pass.bak demonstrates unsafe secret handling.	Advanced Figure 28	Remove plaintext secrets and rotate exposed credentials.

8. SECURITY RECOMMENDATIONS

- Enforce HTTPS/TLS for authentication and session traffic.
- Minimize web-server version and configuration disclosure and disable unnecessary HTTP methods.
- Review and remove development, debugging and administrative artifacts from public web directories.
- Disable anonymous SMB access unless there is a documented business requirement.
- Apply strong password and SSH-key passphrase policies, and rotate exposed credentials.
- Protect private keys and secrets with restrictive permissions and secure secret-management practices.
- Deploy endpoint monitoring and prevention controls for suspicious executables, command shells and remote sessions.
- Apply least privilege across users, services, SMB shares and filesystem access.
- Keep operating systems, encryption software and security tools patched and supported.
- Keep all security testing within explicit authorization and defined scope.

9. SKILLS AND KNOWLEDGE GAINED

- Hands-on network reconnaissance and service enumeration with Nmap.
- Practical web directory enumeration with Gobuster and interpretation of HTTP status codes.
- Packet capture and HTTP POST analysis with Wireshark.
- Recognition of plaintext credential-transmission risk.
- Encrypted-container creation, authentication, mounting and protected-file access using VeraCrypt.

- Basic hexadecimal file inspection and Windows PE header analysis using PE Explorer.
- Network connectivity validation and Windows IP-configuration review.
- Controlled payload-delivery and remote-session concepts in an isolated lab.
- Metasploit handler and Meterpreter session-validation fundamentals.
- Structured penetration-testing methodology using the TryHackMe Basic Pentesting room.
- Credential-security, SMB, SSH private-key and secret-storage analysis.
- Evidence-based cybersecurity reporting, limitation handling and remediation documentation.

10. OVERALL CONCLUSION

Completion of the documented ShadowFox internship tasks provided a progressive practical learning path across multiple cybersecurity domains. The Beginner Level established the fundamentals of reconnaissance, directory enumeration and network-traffic analysis. The Intermediate Level expanded that foundation into data protection, file analysis, Windows PE inspection, network validation and controlled remote-access concepts. The Advanced Level then applied a structured penetration-testing workflow against the TryHackMe Basic Pentesting training machine.

Across the three supplied reports, the work demonstrates practical exposure to identifying exposed services, mapping application resources, analyzing authentication traffic, protecting data with encryption, inspecting executable structures, validating network sessions and chaining configuration and credential weaknesses in an authorized lab. The evidence also reinforces the importance of strong authentication, secure secret storage, least privilege, endpoint monitoring and strict testing authorization.

This report intentionally does not claim real-world professional penetration-testing experience beyond the documented internship and training-lab activities. Where source evidence was unavailable—most notably the wireless task and five Advanced evidence figures—the limitation is explicitly preserved rather than replaced with assumptions.

11. REFERENCES

- ShadowFox Cyber Security Internship task structure supplied with the internship submission.
- TryHackMe – Basic Pentesting — <https://tryhackme.com/room/basicpentestingjt>
- Nmap Documentation / Reference Guide — <https://nmap.org/docs.html>
- Gobuster — <https://github.com/OJ/gobuster>
- THC Hydra — <https://github.com/vanhauser-thc/thc-hydra>
- John the Ripper Documentation — <https://www.openwall.com/john/doc/>
- OpenSSH — <https://www.openssh.com/>
- VeraCrypt Documentation — <https://veracrypt.io/en/Documentation.html>
- Microsoft Learn – Windows Security — <https://learn.microsoft.com/en-us/windows/security/>
- Microsoft Learn – PE Format — <https://learn.microsoft.com/en-us/windows/win32/debug/pe-format>
- Metasploit Documentation — <https://docs.metasploit.com/>
- MITRE ATT&CK; — <https://attack.mitre.org/>

12. FINAL DECLARATION

The activities documented in this final report were performed within authorized and controlled internship/training environments for cybersecurity education and practical skill development. The report uses the supplied task reports and screenshots as the primary evidence record. No unsupported screenshots, credentials, commands, results or activities have been intentionally fabricated.

Submitted By	MANOJ
Program ID	SF-PID-2026-W4884
Domain	Cyber Security
Internship Program	ShadowFox